Complete Observability System (Metrics, Logs & Traces)

ELEVATE LABS DEVOPS INTERNSHIP PROJECT REPORT | PROJECT #3

Domain: DevOps & SRE Engineering	**Timeline:** 2 Weeks Accelerated MVP	**Platform:** Docker Compose Microservices
Author: DevOps Intern	**Affiliation:** Elevate Labs / MSME Govt. of India	**Status:** Fully Implemented & Verified

1. Abstract

Modern cloud-native architectures rely on microservices that communicate asynchronously across distributed networks. While traditional infrastructure monitoring answers whether a host is up or down, it fails to diagnose the root causes of latency bottlenecks, intermittent failures, and distributed dependencies. This project delivers an end-to-end, production-ready Observability Minimum Viable Product (MVP) implementing the **Three Pillars of Observability**: time-series metrics, centralized log streams, and distributed request tracing. By integrating **Prometheus, Grafana Loki, Promtail, Jaeger,** and **Grafana** with an instrumented Python Flask microservice, the solution enables full-lifecycle telemetry correlation from a single unified Grafana dashboard deployed declaratively via Docker Compose.

2. Introduction

Site Reliability Engineering (SRE) defines observability as the ability to infer internal system states purely from external outputs. When an incident occurs in production, operators must rapidly navigate between aggregate symptoms (metrics), historical execution events (logs), and distributed cross-service journeys (traces). This project builds an automated observability ecosystem designed to eliminate monitoring silos. A realistic microservice simulates compute tasks, database latency, and unhandled server errors, outputting correlated telemetry with OpenTelemetry context injection (Trace IDs and Span IDs) directly into the telemetry pipeline.

3. Tools & Technologies Used

Tool / Component	Version / Role	Architectural Function in Project
Prometheus	v2.52.0 (Metrics Engine)	Polls /metrics every 5s; records request rate, status codes, and latency histograms.
Grafana Loki	v3.0.0 (Log Aggregator)	Stores metadata-indexed container logs efficiently in TSDB filesystem chunks without heavy full-text overhead.
Promtail	v3.0.0 (Log Shipper)	Attaches to Docker daemon socket (/var/run/docker.sock) and ships stdout streams to Loki.
Jaeger	v1.57 (Distributed Tracing)	OTLP gRPC collector (port 4317) and trace store; provides waterfall call graph timelines.
Grafana	v10.4.2 (Unified UI)	Pre-provisioned dashboard displaying metrics, logs, and trace links side-by-side with zero manual setup.
OpenTelemetry & Flask	OTel SDK 1.25 / Python 3.11	Auto-instruments HTTP requests, propagates W3C trace context, and provides custom workload spans.
Docker Compose	v2.20+ (Orchestration)	Declarative single-command provisioning (docker compose up -d) for all 6 microservices.

4. Steps Involved in Building the Project

- **Step 1 — Application Design & Telemetry Instrumentation:** Created a Python Flask service (app/app.py) exposing business endpoints (/work, /slow, /error). Instrumented OpenTelemetry TracerProvider with gRPC OTLP exporter to Jaeger and wrapped HTTP routing using FlaskInstrumentor. Integrated prometheus_flask_exporter to automatically expose Prometheus metrics at /metrics. Configured dynamic log record hooks injecting active trace_id and span_id into standard log outputs.
- **Step 2 — Log Aggregation with Promtail & Loki:** Implemented a decoupled logging pattern. The container writes structured logs to stdout. Promtail discovers running containers via the Docker socket, extracts metadata labels (container="flask-observability-app"), and pushes compressed log streams to Grafana Loki via HTTP API push endpoints.
- **Step 3 — Prometheus Metrics Pull Configuration:** Defined declarative scrape jobs in prometheus.yml configured with a 5-second interval targeting the internal Docker DNS name flask-app:5000, tracking request rates, errors, and execution duration quantiles.
- **Step 4 — Distributed Trace Pipeline:** Configured Jaeger All-in-One with OTLP receiver enabled (COLLECTOR_OTLP_ENABLED=true). Instrumented custom internal span operations (compute_job and slow_database_query) to record synthetic database and CPU processing latencies.

- **Step 5 — Automated Grafana Provisioning:** Built automated YAML datasources (datasources.yml) with explicit deterministic UIDs (prometheus, loki, jaeger) and dashboard provider definitions (dashboards.yml). Authored a comprehensive 4-panel dashboard (dashboard.json) correlating throughput, latency, log streams, and trace waterfalls without requiring manual UI configuration.
- **Step 6 — Orchestration & Synthetic Load Testing:** Unified all services within docker-compose.yml with health dependencies and persistent storage mounts. Developed an automated synthetic load generator script (traffic-generator.sh) to simulate realistic production traffic and failure states.

5. Key Results & Observability Insights

During synthetic testing, the system demonstrated seamless cross-pillar telemetry correlation:
• **Real-time Metric Tracking:** PromQL queries accurately isolated spikes in endpoint request rate (sum by (endpoint) (rate(flask_http_request_total[1m]))) and highlighted simulated HTTP 500 error spikes on /error.
• **Trace-to-Log Correlation:** When inspecting high-latency events on /slow (800ms–1500ms), engineers could view the exact trace waterfall in Jaeger and correlate the corresponding log record in Loki using the shared 32-character hexadecimal Trace ID.
• **Zero-Configuration Deployment:** The entire multi-tiered monitoring architecture provisions automatically in under 60 seconds with docker compose up -d.

6. Conclusion

This project successfully designs and deploys a complete, production-ready observability architecture fulfilling all Elevate Labs guidelines. By seamlessly correlating metrics (Prometheus), logs (Loki/Promtail), and distributed traces (Jaeger) into an automated Grafana dashboard, the system eliminates blind spots in microservice architectures and drastically reduces Mean Time to Detection (MTTD) and Mean Time to Resolution (MTTR). The project serves as an exemplary blueprint for modern cloud-native observability and Site Reliability Engineering best practices.